# Lab Tutorial

## Building a GUI App with PySide6

*Eden's Pizza Order App*

## Objective

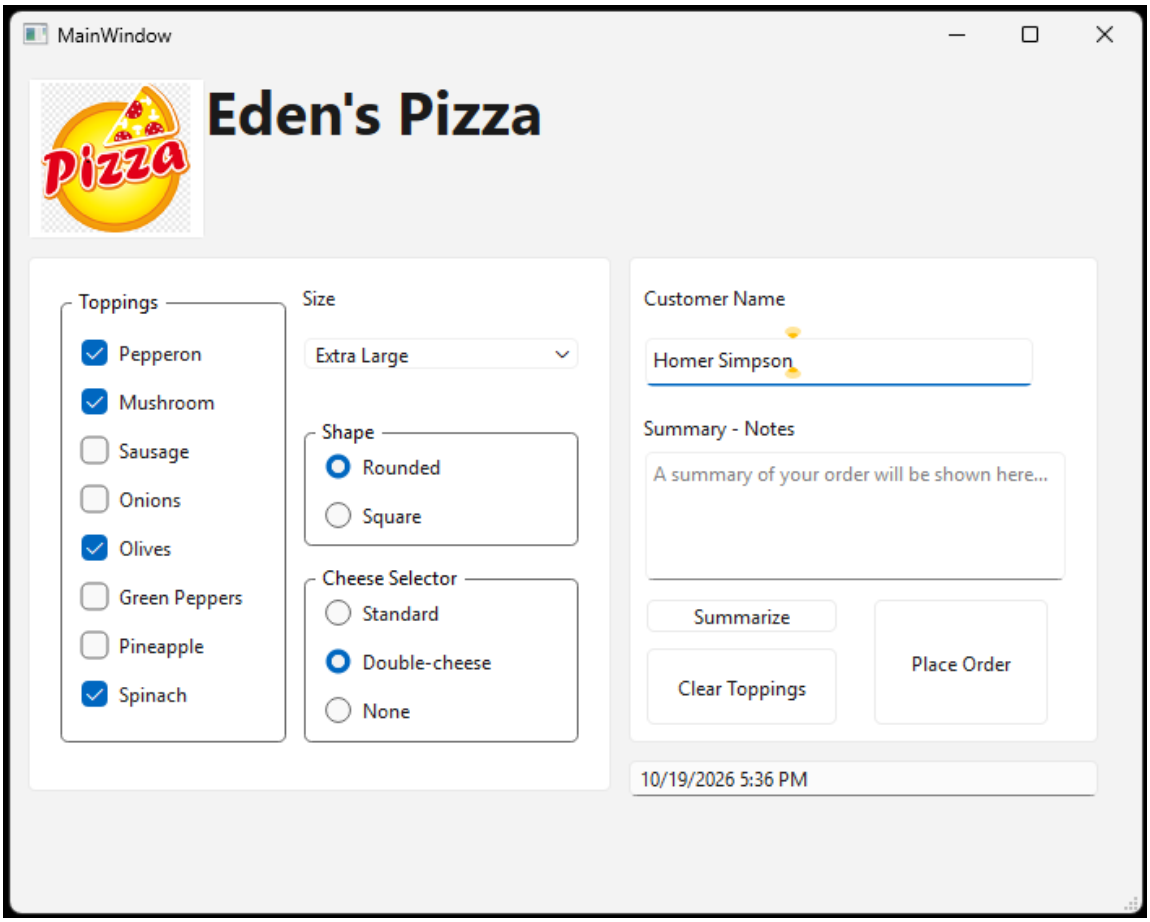
In this lab you will build a complete, runnable GUI desktop application using Python and PySide6. You will follow the recommended two-file architecture used by professional Qt developers:

- pizza_qt_created_gui.py — the auto-generated UI layout file (never edited by hand)
- pizza_app.py — your logic file, containing a single class that inherits from both QMainWindow and Ui_MainWindow

By keeping generated code and your own code in separate files, you can freely redesign the UI in Qt Designer and regenerate pizza_qt_created_gui.py without losing any of your logic.

**Lab Context**

The finished app is Eden's Pizza — a pizza ordering window where a customer selects toppings, shape, cheese, and size, enters a name, and places an order.



## Background — What is PySide6?

PySide6 is Qt's official Python binding. Qt is a mature, cross-platform C++ framework. PySide6 exposes the full Qt 6 library to Python so you can build desktop apps on Windows, macOS, and Linux without changing your code.

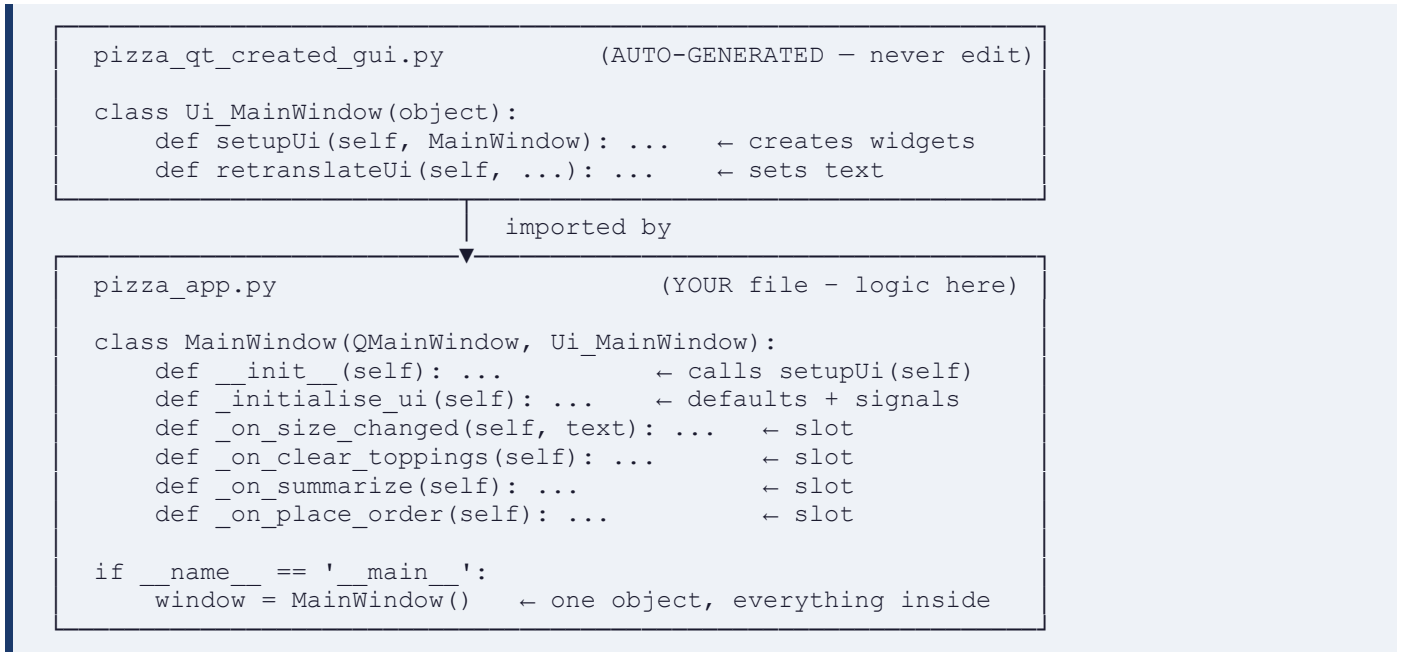
Key vocabulary used in this lab:

| Term | Meaning |
|---|---|
| Widget | Any visible control: button, checkbox, text box, label … |
| Signal | A notification a widget emits automatically when the user interacts with it |
| Slot | A Python method you write that runs in response to a signal |
| Qt Designer | A drag-and-drop GUI builder that saves layouts as .ui (XML) files |
| pyside6-uic | Command-line tool: converts a .ui file into a Python class |
| .ui file | XML description of your window — generated by Qt Designer, never hand-written |
| QMainWindow | The top-level window class that provides menus, toolbars, and status bar |
| QApplication | Manages the event loop and application-wide settings |
| Multiple inheritance | A Python class that inherits from more than one parent class |

## Architecture Overview — Two Files, One Class

This is the pattern you are about to build:

*Architecture diagram — the relationship between the two files:*




MainWindow inherits from two parents simultaneously. This is called multiple inheritance. Python calls both parent __init__ methods via super(). The result: one object that IS a real OS window (QMainWindow) AND has all the widget attributes created by Ui_MainWindow.setupUi().

**Why is this a good approach?**

Your view and logic exist in two different files. That means you may modify the view with pyside6-designer and re-run pyside6-uic, any number of times without affecting your logic. The two-file pattern keeps generated code and your code permanently separate.

## Prerequisites

- Python 3.10 or newer (python --version to check)
- pip available (python -m pip --version)
- A terminal / command prompt
- A code editor (VS Code recommended)
- Starter files: pizza_qt_created_gui.ui and pizza_logo.png
  Visit: https://github.com/VMatosECC/Python-Pizza-App

### Step 1: Set Up the Project

#### 1.1 Create the project folder

*In your terminal:*

```
mkdir eden_pizza
cd eden_pizza
```

Copy pizza_qt_created_gui.ui and pizza_logo.png into the eden_pizza folder.

#### 1.2 Install the required packages

```
pip install PySide6
pip install pyside6-tools
```

**Why two packages?**

PySide6 is the runtime (widgets, events, graphics). pyside6-tools adds pyside6-uic (converts .ui → .py) and pyside6-designer (opens Qt Designer).

#### 1.3 Verify

*Both should print a version number such as 6.9.2*

```
python -c "import PySide6; print(PySide6.__version__)"
pyside6-uic --version
```

**Checkpoint 1**

- Both commands print a version number without error
- eden_pizza/ contains: pizza_qt_created_gui.ui and pizza_logo.png

## Step 2: Understand the .ui File

Open pizza_qt_created_gui.ui in a text editor. It is XML generated by Qt Designer — you never write or edit it. Every <widget> element becomes a Python attribute whose name comes from the name= attribute.

### 2.1 The XML skeleton

```
<?xml version="1.0" encoding="UTF-8"?>
<ui version="4.0">
  <class>MainWindow</class>
  <widget class="QMainWindow" name="MainWindow">
    <widget class="QWidget" name="centralwidget">
      <!-- every child widget defined here -->
    </widget>
    <widget class="QMenuBar"  name="menubar"/>
    <widget class="QStatusBar" name="statusbar"/>
  </widget>
</ui>
```

### 2.2 Widget inventory

| objectName (→ self.name) | Type & purpose |
|---|---|
| labelTopCaption | QLabel — title "Eden's Pizza" (28 pt bold) |
| graphicsView | QGraphicsView — displays the pizza logo |
| groupToppings | QGroupBox — parent of all 8 topping checkboxes |
| chkPepperoni … chkSpinach | QCheckBox × 8 — individual toppings |
| groupShape | QGroupBox — radioShapeRounded, radioShapeSquare |
| groupCheeseSelector | QGroupBox — radioCheeseStandard, radioCheeseDouble, radioCheeseNone |
| comboSize | QComboBox — pizza size choices |
| plainTextCustomerName | QPlainTextEdit — customer enters their name |
| plainTextSummaryNotes | QPlainTextEdit — order summary (written by code) |
| btnClearToppings | QPushButton — clears all checkboxes |
| btnSummarize | QPushButton — writes summary to notes field |
| btnPlaceOrder | QPushButton — shows confirmation dialog |
| dateTimeEdit | QDateTimeEdit — shows current date/time |

**Designer tip**

You can open the .ui file visually any time: run  pyside6-designer pizza_qt_created_gui.ui

## Step 3: Convert the .ui File with pyside6-uic

### 3.1 Run the converter

*Inside the eden_pizza folder:*

```
pyside6-uic pizza_qt_created_gui.ui -o pizza_qt_created_gui.py
```

This creates pizza_qt_created_gui.py with class Ui_MainWindow containing setupUi() and retranslateUi().

### 3.2 The Golden Rule

> **NEVER edit pizza_qt_created_gui.py**
> It is overwritten every time you run pyside6-uic. All your code goes in pizza_app.py. This is the entire point of the two-file architecture.

### 3.3 What setupUi() does

```
# Example of what the generator produces for one widget:
self.chkPepperoni = QCheckBox(self.groupToppings)     # creates the widget
self.chkPepperoni.setObjectName("chkPepperoni")       # names it
self.chkPepperoni.setGeometry(QRect(10, 30, 78, 20))  # positions it
```

After setupUi(self) runs inside MainWindow.__init__(), every widget exists as self.<objectName> — accessible from any method in your MainWindow class.

**Checkpoint 2**

- pizza_qt_created_gui.py exists and contains class Ui_MainWindow
- You have NOT added any code to it
- Your eden_pizza/ folder contains exactly: pizza_qt_created_gui.ui, pizza_qt_created_gui.py, pizza_logo.png

## Step 4: Create pizza_app.py — Imports and Class Declaration

Create a new, empty file called pizza_app.py inside the eden_pizza folder. This is the only file you will write from scratch.

### 4.1 Imports

```
# pizza_app.py  –  Eden's Pizza  –  Application logic

import sys
from pathlib import Path

from PySide6.QtCore    import QDateTime, Qt
from PySide6.QtGui     import QBrush, QColor, QPixmap
from PySide6.QtWidgets import (
    QApplication,
    QCheckBox,
    QGraphicsScene,
    QMainWindow,
    QMessageBox,
)

# Import the generated UI class — do NOT edit that file!
from pizza_qt_created_gui import Ui_MainWindow
```

### 4.2 The MainWindow class declaration

```
class MainWindow(QMainWindow, Ui_MainWindow):
    """The application's main window.

    Inherits from TWO parents:
      QMainWindow   - provides the real OS window (title bar, menus …)
      Ui_MainWindow - provides setupUi() which creates all the widgets
    """
```

This is the multiple inheritance declaration. Python reads it left-to-right: QMainWindow is the primary parent (it provides __init__); Ui_MainWindow is mixed in (it contributes setupUi and retranslateUi).

> **How Python resolves two parents**
>
> When you call super().__init__(parent) inside MainWindow.__init__, Python's Method Resolution Order (MRO) ensures QMainWindow.__init__ runs first, giving you a real window. Ui_MainWindow has no __init__ of its own, so its methods (setupUi, retranslateUi) are simply available as methods of your class.

## Step 5: Write __init__ and the asset path helper

### 5.1 The resource_path() helper

Add this function before the class definition. It finds asset files whether you are running the script normally or inside a PyInstaller .exe:

```
def resource_path(relative: str) -> Path:
    """Return the absolute path to a bundled asset.

    During development:  returns the folder containing this .py file.
    Inside a PyInstaller .exe: returns sys._MEIPASS, the temp folder
    where PyInstaller extracts bundled files at runtime.
    """
    base = getattr(sys, "_MEIPASS", Path(__file__).resolve().parent)
    return Path(base) / relative
```

### 5.2 __init__

*Inside class MainWindow:*

```
    def __init__(self, parent=None):
        # Step A: initialise both parent classes
        super().__init__(parent)

        # Step B: build all the widgets defined in the .ui file.
        # After this call, every widget is available as self.<objectName>.
        self.setupUi(self)   # <── 'self' is both the window AND the receiver

        # Step C: set defaults and wire up signals → slots.
        self._initialise_ui()
```

Notice the argument to setupUi: we pass self, which is the MainWindow instance. Ui_MainWindow.setupUi() expects a QMainWindow as its argument — and that is exactly what self is, thanks to multiple inheritance.

| Line | Purpose |
|---|---|
| `super().__init__(parent)` | Calls QMainWindow.__init__; creates the real OS window |
| `self.setupUi(self)` | Calls Ui_MainWindow.setupUi; populates the window with all widgets |
| `self._initialise_ui()` | Your method: sets defaults, loads image, connects signals |

**Checkpoint 3**

- class MainWindow(QMainWindow, Ui_MainWindow): is declared
- __init__ calls super().__init__, then self.setupUi(self), then self._initialise_ui()
- resource_path() is defined above the class

## Step 6: Write _initialise_ui() — Defaults and Signal Connections

This private method (the leading underscore _ signals 'internal use') sets every widget's starting state and registers the signal → slot connections. It is called once from __init__.

### 6.1 Default widget states

```
    def _initialise_ui(self):
        # Customer name field
        self.plainTextCustomerName.setPlaceholderText(
            "Enter customer name here..."
        )

        # Size combo box
        self.comboSize.addItems(["Small", "Medium", "Large", "Extra Large"])
        self.comboSize.setCurrentIndex(1)  # default: Medium

        # Radio button defaults
        self.radioShapeRounded.setChecked(True)
        self.radioCheeseStandard.setChecked(True)

        # Date/time — show now, hide the spin arrows
        self.dateTimeEdit.setDateTime(QDateTime.currentDateTime())
        self.dateTimeEdit.setButtonSymbols(
            self.dateTimeEdit.ButtonSymbols.NoButtons
        )

        # Summary notes placeholder
        self.plainTextSummaryNotes.setPlaceholderText(
            "A summary of your order will be shown here..."
        )

        # Load the logo
        self._load_logo()
```

### 6.2 Signal → slot connections

Add these lines at the very end of _initialise_ui(), after the _load_logo() call:

```
        # Signal connections — each line says:
        # 'when THIS signal fires, call THIS slot method'
        self.comboSize.currentTextChanged.connect(self._on_size_changed)
        self.btnClearToppings.clicked.connect(self._on_clear_toppings)
        self.btnSummarize.clicked.connect(self._on_summarize)
        self.btnPlaceOrder.clicked.connect(self._on_place_order)
```

**The .connect() pattern**

widget.signal.connect(self.method) registers the connection. Qt calls self.method automatically whenever the signal fires. You never call slot methods directly from your own code — Qt does that for you.

### 6.3 Signal reference

| Widget class | Signal | When fired |
|---|---|---|
| QPushButton | .clicked | When the button is pressed |
| QComboBox | .currentTextChanged | When the selected item changes — passes the new text as a str |
| QCheckBox | .stateChanged | When checked/unchecked — passes Qt.Checked or Qt.Unchecked |
| QRadioButton | .toggled | When selected/deselected — passes True or False |
| QPlainTextEdit | .textChanged | Every time the text content changes |

## Step 7: Write _load_logo() — Displaying an Image

The QGraphicsView is a canvas. To show an image you need three Qt objects working together:

| Object | Role |
|---|---|
| QPixmap | Loads the image file from disk into memory |
| QGraphicsScene | An invisible container that holds graphics items (pixmaps, shapes, text) |
| QGraphicsView | The visible widget that renders a scene — you attach the scene to the view |

```
    def _load_logo(self):
        """Load pizza_logo.png into the QGraphicsView widget."""
        pizza_pixmap = QPixmap(str(resource_path("pizza_logo.png")))
        scene = QGraphicsScene(self.graphicsView)

        if not pizza_pixmap.isNull():        # image loaded successfully
            scene.addPixmap(pizza_pixmap)
            scene.setSceneRect(pizza_pixmap.rect())
            self.graphicsView.setScene(scene)
            self.graphicsView.fitInView(
                scene.sceneRect(),
                Qt.AspectRatioMode.KeepAspectRatioByExpanding,
            )
        else:                                # visible fallback if file missing
            self.graphicsView.setBackgroundBrush(QBrush(QColor(255, 0, 0)))
            self.graphicsView.setScene(scene)
```

**pizza_pixmap.isNull()**

QPixmap.isNull() returns True when the file could not be loaded — wrong path, missing file, or unsupported format. Always check this before using the pixmap so your app does not silently show a blank view.

**Checkpoint 4**

- _initialise_ui() sets placeholder text, combo items, radio defaults, and date/time
- _load_logo() loads the image using resource_path() and adds it to a QGraphicsScene
- All four signal → slot connections are registered in _initialise_ui()
- The red fallback appears if pizza_logo.png is missing (do not crash)

## Step 8: Write the Slot Methods

Each slot is a regular Python method. Qt calls it automatically when the connected signal fires. Name them clearly: _on_<what_happened>.

### 8.1 _on_size_changed — combo box selection changed

```
    def _on_size_changed(self, new_size: str):
        """Slot for comboSize.currentTextChanged.
        Qt passes the newly selected text as new_size automatically.
        """
        self.plainTextSummaryNotes.setPlainText(
            f"Selected Size: {new_size}"
        )
```

The slot accepts new_size: str because currentTextChanged always passes the new text. If your slot signature does not match the signal's payload, Qt raises a runtime error.

### 8.2 _on_clear_toppings — Clear Toppings button

```
    def _on_clear_toppings(self):
        """Uncheck every topping checkbox and clear the summary area."""
        self.plainTextSummaryNotes.clear()
        for checkbox in self.groupToppings.findChildren(QCheckBox):
            checkbox.setChecked(False)
```

**findChildren(QCheckBox)**

groupToppings.findChildren(QCheckBox) returns a list of every QCheckBox that is a descendant of groupToppings — all 8 of them. The loop works whether there are 3 toppings or 30; you never need to name them individually.

### 8.3 _on_summarize — Summarize button

```
    def _on_summarize(self):
        """Read every widget state and write a formatted summary."""

        # Collect the text of every CHECKED topping checkbox.
        checked_toppings = ", ".join(
            cb.text()
            for cb in self.groupToppings.findChildren(QCheckBox)
            if cb.isChecked()
        ) or "None"

        # Shape — exactly one radio is always checked.
        shape = "Rounded" if self.radioShapeRounded.isChecked() else "Square"

        # Cheese — three mutually exclusive options.
        if self.radioCheeseStandard.isChecked():
            cheese = "Standard"
        elif self.radioCheeseDouble.isChecked():
            cheese = "Double-cheese"
        else:
            cheese = "None"

        summary = (
            f"Customer Name:     "
            f"{self.plainTextCustomerName.toPlainText()}\n"
            f"Selected Size:     {self.comboSize.currentText()}\n"
            f"Selected Toppings: {checked_toppings}\n"
            f"Selected Shape:    {shape}\n"
            f"Selected Cheese:   {cheese}\n"
```

```
        )
        self.plainTextSummaryNotes.setPlainText(summary)
```

Key widget-reading methods:

| Method call | Returns |
|---|---|
| `plainTextCustomerName.toPlainText()` | Returns what the customer typed |
| `comboSize.currentText()` | Returns the selected size as a string |
| `radioShapeRounded.isChecked()` | Returns True if Rounded is selected |
| `cb.isChecked()` | Returns True if this checkbox is ticked |
| `cb.text()` | Returns the checkbox label (e.g. 'Pepperoni') |

### 8.4 _on_place_order — Place Order button

```
    def _on_place_order(self):
        """Show a confirmation dialog before placing the order."""
        msg = QMessageBox(self)          # 'self' centres the dialog on this window
        msg.setWindowTitle("Place Order")
        msg.setText("Are you sure you want to place this order?")
        msg.setInformativeText(self.plainTextSummaryNotes.toPlainText())
        msg.setStandardButtons(QMessageBox.Yes | QMessageBox.No)
        msg.setDefaultButton(QMessageBox.No)

        if msg.exec() == QMessageBox.Yes:
            print("Order placed!\n",
                  self.plainTextSummaryNotes.toPlainText())
        else:
            print("Order cancelled.")
```

msg.exec() is a blocking call — it opens the dialog and waits until the user clicks a button. It returns the button's value (QMessageBox.Yes or QMessageBox.No) which you compare directly.

**Checkpoint 5**

- All four slot methods are defined inside class MainWindow
- _on_summarize() reads 5 different widget states and writes a formatted f-string
- _on_place_order() passes 'self' to QMessageBox so the dialog centres on the window
- _on_clear_toppings() uses findChildren() — not 8 individual setChecked calls

## Step 9: Add the Entry Point and Run

### 9.1 The main block

Add this at the bottom of pizza_app.py, outside the class:

```
# ── Entry point ──────────────────────────────────────────────────
if __name__ == "__main__":
    app = QApplication(sys.argv)  # must exist before any widget is created
    window = MainWindow()         # __init__ → setupUi → _initialise_ui
    window.show()
    sys.exit(app.exec())
```

The two files pattern is effective and simple because MainWindow.__init__ handles everything internally. The caller just creates one object and shows it.

### 9.2 Run the app

```
cd eden_pizza
python pizza_app.py
```

The Eden's Pizza window should appear. Test every feature:

- Select toppings → click Summarize → verify they appear in the notes
- Change the size combo → verify notes update immediately
- Click Clear Toppings → all checkboxes unchecked, notes cleared
- Click Place Order → confirmation dialog appears with summary text
- Click Yes → terminal prints the order; click No → terminal prints 'cancelled'
- Close the window → program exits cleanly (no error in terminal)

**ModuleNotFoundError: No module named 'pizza_qt_created_gui'**

You are running python from the wrong folder. cd into eden_pizza first. pizza_qt_created_gui.py must be in the same directory as pizza_app.py.

**Red box instead of logo / image not showing**

pizza_logo.png is not in the eden_pizza folder. Copy it in. The code uses resource_path() to find it relative to pizza_app.py.

## Step 10: Package the App as a Standalone .exe

PyInstaller bundles your Python script, the Python interpreter, PySide6, and all assets into a single .exe that runs on any Windows machine — even one without Python installed.

### 10.1 Install PyInstaller

```
pip install pyinstaller
```

### 10.2 Build command

*Run from inside eden_pizza/ (^ continues the line on Windows):*

```
pyinstaller --onefile ^
            --windowed ^
            --add-data "pizza_logo.png;." ^
            --add-data "pizza_qt_created_gui.py;." ^
            --name EdensPizza ^
            pizza_app.py
```

| Flag | Effect |
|---|---|
| `--onefile` | Bundle everything into a single EdensPizza.exe |
| `--windowed` | Hide the terminal window when the .exe runs (GUI app) |
| `--add-data "pizza_logo.png;."` | Include the logo; place it in the bundle root |
| `--add-data "pizza_qt_created_gui.py;."` | Include the generated UI module alongside pizza_app.py |
| `--name EdensPizza` | Name the output file EdensPizza.exe |
| `pizza_app.py` | The entry point — your logic file |

**Why include pizza_qt_created_gui.py?**

pizza_app.py does 'from pizza_qt_created_gui import Ui_MainWindow' at runtime. PyInstaller must bundle the imported module. --add-data places it where Python can find it inside the .exe's temporary extraction folder.

### 10.3 The resource_path() function — why it is needed

When the --onefile .exe starts, PyInstaller extracts all bundled files to a temporary folder. The path to that folder is stored in sys._MEIPASS. The resource_path() function you wrote in Step 5 handles both cases:

```
def resource_path(relative: str) -> Path:
    base = getattr(sys, "_MEIPASS",       # inside .exe: temp extraction folder
                   Path(__file__).resolve().parent)  # during development
    return Path(base) / relative
```

### 10.4 Run the .exe

```
dist\EdensPizza.exe
```

**Checkpoint 6**

- pyinstaller command completes without error
- dist/EdensPizza.exe exists
- Double-clicking EdensPizza.exe opens the pizza window with the logo visible
- All buttons and controls behave exactly as when running with python pizza_app.py

## Step 11: Complete File Listing — pizza_app.py

Below is the complete, annotated pizza_app.py. Every line should now make sense.

```
# -*- coding: utf-8 -*-
# pizza_app.py  -  Eden's Pizza  -  Application logic

import sys
from pathlib import Path

from PySide6.QtCore    import QDateTime, Qt
from PySide6.QtGui     import QBrush, QColor, QPixmap
from PySide6.QtWidgets import (
    QApplication, QCheckBox, QGraphicsScene,
    QMainWindow, QMessageBox,
)
from pizza_qt_created_gui import Ui_MainWindow

def resource_path(relative):
    base = getattr(sys, '_MEIPASS', Path(__file__).resolve().parent)
    return Path(base) / relative


class MainWindow(QMainWindow, Ui_MainWindow):

    def __init__(self, parent=None):
        super().__init__(parent)
        self.setupUi(self)
        self._initialise_ui()
```

```
    def _initialise_ui(self):
        self.plainTextCustomerName.setPlaceholderText('Enter customer name here...')
        self.comboSize.addItems(['Small', 'Medium', 'Large', 'Extra Large'])
        self.comboSize.setCurrentIndex(1)
        self.radioShapeRounded.setChecked(True)
        self.radioCheeseStandard.setChecked(True)
        self.dateTimeEdit.setDateTime(QDateTime.currentDateTime())
        self.dateTimeEdit.setButtonSymbols(
            self.dateTimeEdit.ButtonSymbols.NoButtons)
        self.plainTextSummaryNotes.setPlaceholderText(
            'A summary of your order will be shown here...')
        self._load_logo()
        self.comboSize.currentTextChanged.connect(self._on_size_changed)
        self.btnClearToppings.clicked.connect(self._on_clear_toppings)
        self.btnSummarize.clicked.connect(self._on_summarize)
        self.btnPlaceOrder.clicked.connect(self._on_place_order)

    def _load_logo(self):
        pixmap = QPixmap(str(resource_path('pizza_logo.png')))
        scene  = QGraphicsScene(self.graphicsView)
        if not pixmap.isNull():
            scene.addPixmap(pixmap)
            scene.setSceneRect(pixmap.rect())
            self.graphicsView.setScene(scene)
            self.graphicsView.fitInView(
                scene.sceneRect(),
                Qt.AspectRatioMode.KeepAspectRatioByExpanding)
        else:
            self.graphicsView.setBackgroundBrush(QBrush(QColor(255,0,0)))
            self.graphicsView.setScene(scene)

    def _on_size_changed(self, new_size):
        self.plainTextSummaryNotes.setPlainText(f'Selected Size: {new_size}')

    def _on_clear_toppings(self):
        self.plainTextSummaryNotes.clear()
        for cb in self.groupToppings.findChildren(QCheckBox):
            cb.setChecked(False)

    def _on_summarize(self):
        toppings = ', '.join(
            cb.text() for cb in self.groupToppings.findChildren(QCheckBox)
            if cb.isChecked()) or 'None'
        shape  = 'Rounded' if self.radioShapeRounded.isChecked() else 'Square'
        cheese = ('Standard'     if self.radioCheeseStandard.isChecked() else
                  'Double-cheese' if self.radioCheeseDouble.isChecked()   else 'None')
        self.plainTextSummaryNotes.setPlainText(
            f'Customer Name:     {self.plainTextCustomerName.toPlainText()}\n'
            f'Selected Size:     {self.comboSize.currentText()}\n'
            f'Selected Toppings: {toppings}\n'
            f'Selected Shape:    {shape}\n'
            f'Selected Cheese:   {cheese}\n'
        )

    def _on_place_order(self):
        msg = QMessageBox(self)
        msg.setWindowTitle('Place Order')
        msg.setText('Are you sure you want to place this order?')
        msg.setInformativeText(self.plainTextSummaryNotes.toPlainText())
        msg.setStandardButtons(QMessageBox.Yes | QMessageBox.No)
        msg.setDefaultButton(QMessageBox.No)
        if msg.exec() == QMessageBox.Yes:
```

```
            print('Order placed!\n', self.plainTextSummaryNotes.toPlainText())
        else:
            print('Order cancelled.')


if __name__ == '__main__':
    app    = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    sys.exit(app.exec())
```

## Key Concepts Summary

- Multiple inheritance: class MainWindow(QMainWindow, Ui_MainWindow) gives one class the features of both parents
- setupUi(self) is called with self because self is the QMainWindow that owns the widgets
- The two-file rule: pizza_qt_created_gui.py is auto-generated; pizza_app.py is yours — keep them separate forever
- Signals fire automatically; you register handlers with widget.signal.connect(self.method)
- Slots are ordinary Python methods; their signature must match the signal's payload
- findChildren(QCheckBox) iterates all descendant checkboxes without naming each one
- resource_path() resolves asset paths correctly both during development and inside a PyInstaller .exe
- sys._MEIPASS is the temp folder a --onefile .exe uses at runtime; getattr with a fallback handles both cases
- QMessageBox.exec() blocks until the dialog is dismissed, then returns the button value
- PyInstaller --add-data must include every non-Python file and every imported Python module that is not auto-detected

## Self-Assessment Checklist

Check each item when you can do it confidently without notes:

| ✓ | I can … |
|---|---|
| ☐ | Explain why the two-file architecture is preferred over putting logic inside Ui_MainWindow |
| ☐ | Write the class declaration for multiple inheritance: class MainWindow(QMainWindow, Ui_MainWindow) |
| ☐ | Explain what super().__init__(parent) does and why it must be called first |
| ☐ | Explain why setupUi(self) passes self, not a separate window object |
| ☐ | Describe the three steps inside __init__: super().__init__, setupUi, _initialise_ui |
| ☐ | Install PySide6 and pyside6-tools and verify both with --version |
| ☐ | Run pyside6-uic to convert a .ui file to a Python class |
| ☐ | Explain the Golden Rule: never edit the auto-generated file |
| ☐ | Set placeholder text on QPlainTextEdit, populate QComboBox, set default radio buttons |
| ☐ | Load and display an image using QPixmap, QGraphicsScene, and QGraphicsView.fitInView() |
| ☐ | Write resource_path() and explain why sys._MEIPASS is checked first |
| ☐ | Connect a button's clicked signal to a slot method using .connect() |
| ☐ | Write a slot that accepts a str payload from currentTextChanged |
| ☐ | Use findChildren(QCheckBox) to iterate all checkboxes inside a group box |
| ☐ | Read QComboBox selection with currentText() and QRadioButton state with isChecked() |
| ☐ | Read QPlainTextEdit content with toPlainText() |
| ☐ | Build a multi-line summary string from multiple widget values using f-strings |
| ☐ | Show a modal QMessageBox with Yes/No buttons and detect which was pressed |
| ☐ | Explain why passing self to QMessageBox() centres the dialog on the window |
| ☐ | Write the minimal entry point: QApplication → MainWindow() → show() → exec() |
| ☐ | Build a --onefile PyInstaller .exe with --windowed and --add-data |
| ☐ | Explain what --add-data does and why both pizza_logo.png and pizza_qt_created_gui.py must be listed |
| ☐ | Run the .exe on a machine without Python and verify it works correctly |